Advanced Robotics

ISSN: 0169-1864 (Print) 1568-5535 (Online) Journal homepage: www.tandfonline.com/journals/tadr20

Connectivity preserving decentralized UAV swarm navigation in obstacle-laden environments without explicit communication

Thiviyathinesvaran Palani, Hiroaki Fukushima & Shunsuke Izuhara

To cite this article: Thiviyathinesvaran Palani, Hiroaki Fukushima & Shunsuke Izuhara (2025) Connectivity preserving decentralized UAV swarm navigation in obstacle-laden environments without explicit communication, Advanced Robotics, 39:6, 305-321, DOI: 10.1080/01691864.2025.2478920

To link to this article: https://doi.org/10.1080/01691864.2025.2478920

Published online: 25 Mar 2025.

Submit your article to this journal

Article views: 136

View related articles

View Crossmark data

Citing articles: 4 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tadr20

ADVANCED ROBOTICS
2025, VOL. 39, NO. 6, 305–321
https://doi.org/10.1080/01691864.2025.2478920

FULL PAPER

Connectivity preserving decentralized UAV swarm navigation in obstacle-laden environments without explicit communication

Thiviyathinesvaran Palani [a], Hiroaki Fukushima [a] and Shunsuke Izuhara [b]

[a]Graduate School of Engineering, Kyoto University of Advanced Science, Kyoto, Japan; [b]Faculty of Environmental, Life, Natural Science and Technology, Okayama University, Okayama, Japan

ABSTRACT
This paper presents a novel control method for a group of UAVs in obstacle-laden environments while preserving sensing network connectivity without data transmission between the UAVs. By leveraging constraints rooted in control barrier functions (CBFs), the proposed method aims to overcome the limitations, such as oscillatory behaviors and frequent constraint violations, of the existing method based on artificial potential fields (APFs). More specifically, the proposed method first determines desired control inputs by considering CBF-based constraints rather than repulsive APFs. The desired inputs are then minimally modified by solving a numerical optimization problem with soft constraints. In addition to the optimization-based method, we present a non-optimization-based method without numerical optimization. The effectiveness of the proposed methods is evaluated by extensive simulations to compare the performance of the CBF-based methods with an APF-based approach. Experimental results using real quadrotors are also presented.

ARTICLE HISTORY
Received 5 October 2024
Revised 23 December 2024 and 2 February 2025
Accepted 26 February 2025

KEYWORDS
Leader–follower navigation; non-communicative; control barrier function; connectivity preservation; collision avoidance

1. Introduction

The realm of robotics has witnessed significant advancements through research on cooperative control of multiple robot systems, driving technological advancements for a wide range of applications [1–3]. This paper focuses on the fundamental problem of how to move a group of robots as a whole to a target area. Specifically, we assume that only one of the robots, called the leader, knows the path to the target area. Thus, since each robot in the group has a limited sensing and communication range, the connectivity of the sensing/communication network must be preserved to avoid leaving some robots behind. Furthermore, to move through cluttered environments including narrow spaces without getting stuck, robots need to flexibly change their network topology in a decentralized way while preserving the network connectivity.

To overcome this problem and to deal with environments where no wireless network is available for information exchange, Sakai et al. [4] proposed a method that does not rely on data transmission between robots. This method can change network topology by deactivating certain network links to allow the robots to pass through narrow spaces while enhancing active links in open areas to maintain group cohesion. The method proposed in [5] extended the approach from [4] for 2D environments to cater to multiple UAVs in 3D environments. However, the method proposed in [5] often suffers from oscillatory movements and frequent constraint violations, which could be a serious problem especially for UAVs. To overcome these problems, we aim to develop a new algorithm inspired by control barrier functions (CBFs), which have extensively studied in recent years.

1.1. Related work

CBFs were initially introduced to deal with inequality constraints for collision avoidance [6–12]. Then they were also applied to various problems, including maintaining connectivity [13–15], coverage mission [16], energy autonomy [17, 18], transporting payloads [19, 20], and herding animals that have strayed [21]. However, the effectiveness of CBFs in the control problem focused in this paper has not been fully studied. Since this paper considers the connectivity aspects of a sensing network without relying on data transmission in the presence of obstacles – a departure from existing studies – we incorporate CBF constraints for preserving line-of-sight (LOS) while maintaining maximum and minimum distances between neighboring robots. Another feature of our control problem is the difficulty of determining desired control inputs, given the dense and complex obstacle-laden environments under consideration, particularly

CONTACT Hiroaki Fukushima fukushima.hiroaki@kuas.ac.jp

© 2025 Informa UK Limited, trading as Taylor & Francis Group and The Robotics Society of Japan

because robots, except for the leader, do not know their destination.

In complicated control problems with multiple CBF constraints, optimization problems of control inputs are not necessarily feasible. To deal with this problem, Breeden and Panagou [22] presented an iterative algorithm to obtain a set of CBFs which ensure that the set composed of those CBFs is a viability domain. Molnar and Ames [23] proposed an algorithmic approach to formulate a single continuously differentiable CBF to capture complex safety specifications by multiple CBFs. In this paper, we directly impose multiple CBF constraints in an optimization process, mirroring the approach taken in many other studies [7, 8, 14, 15, 18, 20, 24]. The infeasibility of the optimization problems is addressed through a classical method using soft constraints [25]. A possible future work is the incorporation of more sophisticated CBF construction algorithms, such as those outlined in [22, 23], into our control method.

While these CBF-based methods resolve a numerical optimization problem at each time step, many control methods for multi-agent systems adopt simple control algorithms without solving numerical optimization [4, 5, 26–30]. These methods determine control inputs through a simple combination of repulsive and attractive actions based on artificial potential fields (APFs) considering constraints among robots and obstacles. However, as suggested in [10], APF-based methods tend to suffer from more oscillations than CBF-based methods. Thus, one important research direction is how to construct simple control algorithms without solving numerical optimizations while taking advantage of CBFs capable of generating smooth behaviors. However, control algorithms that incorporate CBFs into these simple approaches for maneuvering through obstacle-laden environments have not been fully studied.

1.2. Contribution

In this paper, we propose a novel CBF-based control method for coordinating a group of robots in obstacle-laden environments while preserving the connectivity of the sensing network without data transmission between robots. In this method, CBF constraints are constructed to preserve LOS integrity in addition to the maximum and minimum separation distances between neighboring robots. This design is necessary to uphold the connectivity of a sensing network without data transmission in the presence of obstacles, unlike existing CBF control methods [6–21]. Furthermore, unlike the existing method [5] based on APFs, the desired inputs are determined based on the CBF constraints to mitigate oscillatory movements and frequent constraint violations for UAVs.

In addition to the optimization-based method, we present a non-optimization-based method that does not rely on numerical optimization. While CBFs are expected to improve oscillatory behaviors, they entail higher computational costs to solve numerical optimization problems. Thus, it is practically important to investigate simple CBF-based algorithms without solving numerical optimization problems.

We perform extensive simulations to compare the performance of the proposed CBF-based methods against an APF-based method, considering aspects such as constraint violations, computation time, and oscillation of responses. Simulation results show that both the optimization-based and non-optimization-based methods generate fewer oscillatory responses and constraint violations than an APF-based method. Experimental results using real quadrotors are also shown.

A preliminary version of this work was presented in [31]. In this paper, we present a new non-optimization-based method that does not necessitate the iterative modification of input, as outlined in [31]. The optimization-based method is also introduced alongside intensive simulation results to compare the performance of the proposed CBF-based methods (the optimization-based and non-optimization-based methods) against an APF-based method.

2. Problem formulation

We consider a group of N homogeneous robots that operate in a 3D environment containing stationary obstacles. Our control algorithm is constructed based on the assumption that the movement of the ith robot ($i = 1, 2, \ldots, N$) is described by the following second-order system:

$$\begin{aligned} \dot{x}_i &= v_i, \\ \dot{v}_i &= u_i, \quad \|u_i\| \leq \eta, \end{aligned} \tag{1}$$

where $x_i \in \mathbb{R}^n, v_i \in \mathbb{R}^n$, and $u_i \in \mathbb{R}^n$ are the position, velocity, and control input of robot i, respectively. The inclusion of a positive constant $\eta > 0$ in the input constraint accounts for hardware limitations.

We assume that a sufficient condition for collision avoidance between robots i and j is given as follows:

$$\|x_i - x_j\| \geq \underline{d}_c, \quad \forall j \in \mathcal{V} \setminus \{i\}, \tag{2}$$

where $\mathcal{V} := \{1, 2, \ldots, N\}$ is the set of indices of all robots. Although a robot is modeled as a point in (1), the parameter $\underline{d}_c$ in (2) must be determined by taking into account the size of the actual robots. We also assume that a

sufficient condition for obstacle avoidance is given as

$$\|x_i - x_o\| \geq \underline{d}_o, \quad \forall\, x_o \in \mathcal{O}, \tag{3}$$

where $\mathcal{O}$ is a set of all points on obstacles in the workspace.

To describe the sensing model, we first define the line segment joining p and q as

$$\mathcal{L}(p,q) := \{(1-\lambda)p + \lambda q,\ \forall\, \lambda \in [0,1]\}. \tag{4}$$

We then assume that robot i can sense the position and velocity of robot $j \in \mathcal{V} \setminus \{i\}$ if the following conditions are satisfied

$$\|x_j - x_i\| \leq d_s, \tag{5}$$

$$\|q - x_o\| \geq \underline{d}_{ls}, \quad \forall\, q \in \mathcal{L}(x_i, x_j), \quad \forall\, x_o \in \mathcal{O}. \tag{6}$$

The condition in (5) implies that the maximum sensing range is given by a positive number d_s. The condition in (6) implies that the distance from $\mathcal{L}(x_i, x_j)$ to each obstacle is at least the minimum clearance $\underline{d}_{ls}$, ensuring that obstacles do not interrupt the LOS between robots i and j. We also assume that robot i can detect a point on an obstacle $x_o \in \mathcal{O}$, if

$$\|x_o - x_i\| \leq d_s, \tag{7}$$

$$\|x_o - x_i\| \leq \|q - x_i\|, \quad \forall\, q \in \bar{\mathcal{L}}(x_i, x_o) \cap \mathcal{O}, \tag{8}$$

where $\bar{\mathcal{L}}(x_i, x_o) := \{(1-\lambda)x_i + \lambda x_o, \forall \lambda \geq 0\}$. While the set $\mathcal{L}(x_i, x_o)$ includes only points between x_i and x_o, the set $\bar{\mathcal{L}}(x_i, x_o)$ includes points behind x_o along the line from x_i to x_o, in addition to the points in $\mathcal{L}(x_i, x_o)$. Thus, the condition in (8) implies that no other point on an obstacle is closer to x_i than x_o on $\mathcal{L}(x_i, x_o)$. We denote $\mathcal{O}_i(t)$ as the set of points on obstacles detected by robot i at time t.

In terms of the sensing described above, we represent the network topology of the multi-robot system using a graph $\mathcal{G}_s(x(t)) = (\mathcal{V}, \mathcal{E}_s(x(t)))$ where $x := [x_1, x_2, \ldots, x_N]$. We denote $\mathcal{V}$ and $\mathcal{E}_s(x(t))$ as the node set and edge set, respectively. The elements of $\mathcal{E}_s(x(t))$ are pairs of robot indices that can sense each other's positions at time t. A connected graph $\mathcal{G}_s$ implies that, for every pair of nodes, there exists a path from one node to the other.

We assume that a target path is given only to the leader of the N robots, whose index is set as N without loss of generality. The remaining robots $i\,(= 1, 2, \ldots, N-1)$ are called followers. Each follower is presumed to be unaware of whether another robot serves as the leader. If $\mathcal{G}_s$ is connected, a pathway exists between the leader and each follower. Thus, since the maximum length of each link of $\mathcal{G}_s$ is kept to no more than a given finite value d_s, each follower is forced to follow the leader at a certain distance (at most $(N-1)d_s$). Furthermore, reducing the length of the link to below d_s allows for a decrease in the distance between the leader and a follower. Thus, in this paper we aim to preserve the connectivity of the following subgraph of $\mathcal{G}_s(x(t))$,

$$\begin{aligned} &\mathcal{G}_m(x(t)) := (\mathcal{V}, \mathcal{E}_m(x(t))), \\ &\mathcal{E}_m(x(t)) := \{(i,j) \in \mathcal{E}_s(x(t)) \mid \|x_i(t) - x_j(t)\| \leq \bar{d}_m\}, \end{aligned} \tag{9}$$

where $\underline{d}_c < \bar{d}_m < d_s$. Then, the neighbors of robot i are defined as

$$\mathcal{N}_i(x(t)) := \{j \mid (i,j) \in \mathcal{E}_m(x(t))\}. \tag{10}$$

From the definition of $\mathcal{G}_m(x(t))$, the connectivity of $\mathcal{G}_s$ is preserved if that of $\mathcal{G}_m(x(t))$ is preserved.

We assume that $\mathcal{G}_m$ is connected at the initial time $t = 0$. Thus, the simplest way to preserve connectivity is to control the robots such that the edges of $\mathcal{G}_m$ at $t = 0$ are not lost. However, in obstacle-laden environments, it is often necessary to change the network topology to facilitate navigation through a narrow space. Thus, it is necessary to select edges to be maintained such that the connectivity of $\mathcal{G}_m$ is preserved. To describe the edges to be preserved, we define the symmetric indicator function $\sigma_{ij}(t) = \sigma_{ji}(t) \in \{0, 1\}$. If $\sigma_{ij} = 1$, there will be an effort to preserve the edge (i,j). Essentially, robot i aims to preserve the link to robot j in the following set:

$$\mathcal{N}_i^{\sigma}(x(t)) := \{j \in \mathcal{N}_i(t) \mid \sigma_{ij}(t) = 1\}. \tag{11}$$

We also define the following subgraph of $\mathcal{G}_m(x(t))$ as

$$\mathcal{G}_\sigma(x(t)) := (\mathcal{V}, \mathcal{E}_\sigma(x(t))), \tag{12}$$

$$\mathcal{E}_\sigma(x(t)) := \{(i,j) \in \mathcal{E}_m(x(t)) \mid \sigma_{ij}(t) = 1\}. \tag{13}$$

By ensuring that σ_{ij} is set in a manner that renders $\mathcal{G}_\sigma$ connected, the connectivity of $\mathcal{G}_m$ is maintained by orchestrating the movement of the robots to avoid any loss of edges within $\mathcal{G}_\sigma$.

We propose an algorithm to compute u_i in a manner that upholds the connectivity of $\mathcal{G}_m$, while adhering to the collision avoidance conditions in (2) and (3). However, when applied to UAVs, it is difficult to avoid constraint violations completely due to modeling errors and disturbances, especially when multiple constraints are imposed severely. Thus, for the constants satisfying $d_m < \bar{d}_m$, $d_c > \underline{d}_c$, $d_o > \underline{d}_o$, and $d_{ls} > \underline{d}_{ls}$, we control the robots so as to satisfy the following constraints:

(i) The maximum distance constraint

$$\|x_i - x_j\| \leq d_m, \quad \forall\, j \in \mathcal{N}_i^{\sigma}(t). \tag{14}$$

(ii) The inter-robot collision avoidance constraint

$$\|x_i - x_j\| \geq d_c, \quad \forall j \in \mathcal{N}_i(t). \tag{15}$$

(iii) The obstacle avoidance constraint

$$\|x_i - x_o\| \geq d_o, \quad \forall x_o \in \mathcal{O}_i(t). \tag{16}$$

(iv) The LOS preservation constraint for each $j \in \mathcal{N}_i^{\sigma}(t)$

$$\|q - x_o\| \geq d_{ls}, \quad \forall x_o \in \mathcal{O}, \quad \forall q \in \mathcal{L}(x_i, x_j). \tag{17}$$

If all of these constraints are satisfied at time t, the state is referred to as Normal mode. On the other hand, if one or more constraints are violated at t, we use a different control law to recover from the constraint violation, referred to as Recovery mode. In this paper, we focus mainly on Normal mode and propose new control methods based on CBFs.

3. Constraints based on CBFs

In this section, we derive CBF-based constraints used in the proposed methods in the same way as in [7], after summarizing the motivation to use them instead of APFs.

3.1. Motivation

To explain the motivation to use CBF-based constraints, we describe a limitation of the APF-based approach used in [4, 5].

For the constraints in (14) and (15), the following repulsive APF is used in [4, 5],

$$\Psi_i^{\mathrm{col}}(x) = \sum_{j \in \mathcal{N}_i^{\sigma}} \Phi^{\mathrm{col}}(\|x_i - x_j\|),$$

$$\Phi^{\mathrm{col}}(z) := \frac{(z-d_r)^2(d_m - z)}{(d_m-d_c)^2(z-d_c) + (d_r-d_c)^2(d_m-z)/\kappa_1} + \frac{(z-d_c)(z-d_r)^2}{(d_m-d_c)^2(d_m-z) + (z-d_c)(d_m-d_r)^2/\kappa_2} \tag{18}$$

where κ_1 and κ_2 are design parameters whose values are equivalent to $\Phi^{\mathrm{col}}(z)$ at $z = d_c$ and $z = d_m$, respectively. Figure 1 illustrates an example of $\Phi^{\mathrm{col}}(z)$ for $d_c = 0.3, d_r = 0.7, d_m = 1.0, \kappa_1 = \kappa_2 = 10$. As the example shows, $\Phi^{\mathrm{col}}(z)$ at the minimum value at the desired relative distance d_r, and monotonically increases as z approaches the maximum allowable distance d_m or the minimum allowable distance d_c. Similarly, for the constraints in (16) and (17), repulsive APFs increase as the distance decreases towards the minimum limits d_o and d_{ls}, respectively.

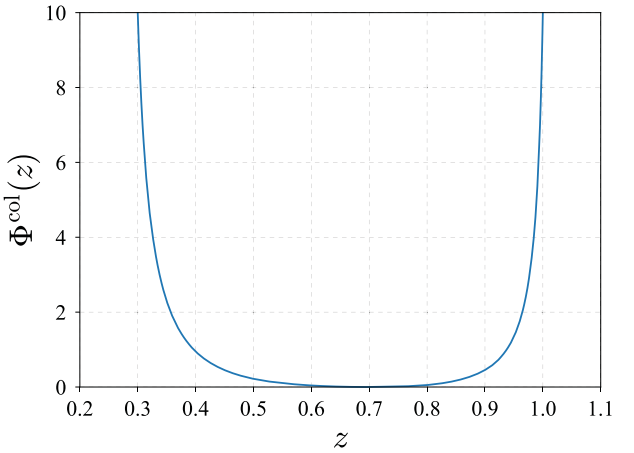

Figure 1. Example of $\Phi^{\mathrm{col}}(z)$ ($d_c = 0.3, d_r = 0.7, d_m = 1.0$).

A limitation of the APF-based approach is that almost no action is taken when the distance is far from the maximum and minimum limits, irrespective of how high the robot's speed in advancing towards a limit. As a result, if the velocity aimed at reaching a constraint boundary becomes too high, the constraint is violated even if the maximum allowable repulsive acceleration is applied near the constraint boundary to avoid violation. Furthermore, the repulsive acceleration is large when the distance is close to the maximum and minimum limits, irrespective of how slow the robot is moving towards the constraint boundary. Thus, when the robot approaches a constraint boundary slowly, an excessively strong repulsive force is applied, which causes oscillatory behaviors. Although a damping filter was used to suppress oscillations in [5], this approach does not address the root cause of the problem. In other words, such a damping filter increases the risk of constraint violations by potentially eliminating essential actions required to avoid constraint violations. On the other hand, CBF-based methods consider velocity constraints depending on the proximity to their limits, as described in Section 3.2.

3.2. Formulation of CBF constraints

For a constraint $h(x) \geq 0$ to the position vector of robots, x, CBF-based methods consider the following derivative condition,

$$\dot{h}(x, v) = \nabla h(x)^T \dot{x} = \nabla h(x)^T v \geq -\kappa(h(x)), \tag{19}$$

where $v := [v_1, v_2, \ldots, v_N]^T$, and $\kappa(\cdot)$ is an extended class $\mathcal{K}$ function. In other words, the lower limit of $\dot{h}(x, v)$ is 0 when the constraint reaches its minimum limit, $h(x) = 0$, and diminishes further as the distance from the

limit increases. Thus, this constraint aims to prevent high speeds of $h(x)$ towards the limit, $h(x) = 0$.

The constraint in (15) for collision avoidance between robots can be described as

$$h_{ij}^c(x) := \|x_i - x_j\| - d_c \geq 0, \quad \forall j \in \mathcal{N}_i. \tag{20}$$

To keep this constraint, we can obtain the following derivative condition in a manner similar to (19) for $\kappa(h_{ij}^c) = \alpha_c h_{ij}^c$.

$$G_{ij}^c(x, v) := \dot{h}_{ij}^c(x, v) + \alpha_c h_{ij}^c(x) \geq 0, \quad \forall j \in \mathcal{N}_i, \tag{21}$$

where

$$\dot{h}_{ij}^c(x, v) = \left(\nabla_{x_i} h_{ij}^c(x)\right)^T v_i + \left(\nabla_{x_j} h_{ij}^c(x)\right)^T v_j. \tag{22}$$

Several approaches [7, 12, 32, 33] are available to deal with derivative conditions which do not include the control input as in (21). In this paper, we adopt the approach in [7], considering a safety margin determined by the maximum braking acceleration, as described below. Comparing the control performance with other approaches is a potential area for future exploration.

Considering a safety margin based on the maximum relative deceleration 2η, the constraint in (20) is modified as

$$h_{ij}^c(x) + \bar{v}_{ji}(t) T_{ij} + 2 \int_0^{T_{ij}} \eta \tau \, \mathrm{d}\tau \geq 0, \tag{23}$$

where $\bar{v}_{ji}(t) := \frac{x_{ji}^T(t)}{\|x_{ji}(t)\|} v_{ji}(t)$, $x_{ji} := x_i - x_j$, $v_{ji} := v_i - v_j$, and $T_{ij} := \frac{0 - \bar{v}_{ji}(t)}{2\eta}$.

When both robots i and j apply the maximum deceleration, the relative velocity $\bar{v}_{ji}(t + T_{ij}) = \bar{v}_{ji}(t) - 2\eta T_{ij}$ diminishes to 0. From (23), a constraint $\bar{h}_{ij}^c(x, v) \geq 0$ is obtained, where

$$\bar{h}_{ij}^c(x, v) := \sqrt{4\eta h_{ij}^c(x)} + \frac{x_{ji}^T}{\|x_{ji}\|} v_{ji}. \tag{24}$$

Then, a derivative condition can be derived for (24) as follows,

$$x_{ji}^T u_{ji} + b_{ij}^c \geq 0, \tag{25}$$

where

$$b_{ij}^c := \alpha_c (\bar{h}_{ij}^c)^3 \|x_{ji}\| - (\bar{v}_{ji})^2 + \|v_{ji}\|^2 + \sqrt{\frac{\eta}{h_{ij}^c}} v_{ji}^T x_{ji}. \tag{26}$$

Based on (25), the following distributed constraint is proposed in [7],

$$x_{ji}^T u_i + b_{ij}^c/2 \geq 0, \tag{27}$$

$$-x_{ji}^T u_j + b_{ij}^c/2 \geq 0. \tag{28}$$

Thus, the constraint for u_i to adhere to the constraint in (15) is described as

$$A_{ij}^c u_i + b_{ij}^c/2 \geq 0, \quad \forall j \in \mathcal{N}_i, \tag{29}$$

where $A_{ij}^c := x_{ji}^T$. Similarly, a CBF-based constraint to maintain the constraint in (14) is described as

$$G_{ij}^m(x, v) := \dot{h}_{ij}^m(x, v) + \alpha_m h_{ij}^m(x) \geq 0, \quad \forall j \in \mathcal{N}_i^\sigma, \tag{30}$$

for $h_{ij}^m(x) := d_m - \|x_i - x_j\|$ and a positive constant α_m. A similar constraint to (29) is derived as

$$A_{ij}^m u_i + b_{ij}^m/2 \geq 0, \quad \forall j \in \mathcal{N}_i^\sigma, \tag{31}$$

where $A_{ij}^m := -x_{ji}^T$, and

$$b_{ij}^m := \alpha_m (\bar{h}_{ij}^m)^3 \|x_{ji}\| + (\bar{v}_{ji})^2 - \|v_{ji}\|^2 - \sqrt{\frac{\eta}{h_{ij}^m}} v_{ji}^T x_{ji},$$

$$\bar{h}_{ij}^m(x, v) := \sqrt{4\eta h_{ij}^m(x)} - \frac{x_{ji}^T}{\|x_{ji}\|} v_{ji}. \tag{32}$$

For the obstacle avoidance constraint in (16), a CBF-based constraint is described as

$$G_{io}^{ob}(x_i, x_o, v_i) := \dot{h}_{io}^{ob}(x_i, x_o, v_i) + \alpha_{ob} h_{io}^{ob}(x_i, x_o) \geq 0, \tag{33}$$

for $h_{io}^{ob}(x_i, x_o) := \|x_i - x_o\| - d_o$ and a positive constant α_{ob}. A modified constraint including u_i is obtained as

$$A_{io}^{ob} u_i + b_{io}^{ob} \geq 0, \tag{34}$$

where $A_{io}^{ob} := x_{oi}^T$, $x_{oi} := x_i - x_o$, and

$$b_{io}^{ob} := \alpha_{ob} (\bar{h}_{io}^{ob})^3 \|x_{oi}\| - \frac{(v_i^T x_{oi})^2}{\|x_{oi}\|^2} + \|v_i\|^2 + \sqrt{\frac{2\eta}{h_{io}^{ob}}} v_i^T x_{oi},$$

$$\bar{h}_{io}^{ob}(x_i, x_o, v_i) := \sqrt{2\eta h_{io}^{ob}(x)} + \frac{x_{oi}^T}{\|x_{oi}\|} v_i. \tag{35}$$

For implementation, it is not realistic to consider the constraint in (34) for each $x_o \in \mathcal{O}_i(t)$. Therefore, we consider the constraints in (34) for the nearest obstacle points from robot i as described in Section 4.2.

In Section 3.3, we derive a CBF-based constraint to keep the constraint in (17) for LOS preservation.

3.3. Constraints for LOS preservation

In the same way as in the case of (34), constraints for LOS preservation are derived for the nearest obstacle point x_{ijo}^* from the LOS line.

The position x_q of the point q projected from x^*_{ijo} onto the LOS between i and j, as shown in Figure 2, is described as

$$x_q := (1-\lambda^*)x_i + \lambda^* x_j, \quad \lambda^* := \frac{(x_i - x^*_{ijo})^T x_{ji}}{\|x_{ji}\|^2}, \tag{36}$$

where x^*_{ijo} is the nearest obstacle point from $\mathcal{L}(x_i, x_j)$. For $x_{oq} := x_q - x^*_{ijo}$, $h^{ls}_{ijo}(x, x^*_{ijo}) := \|x_{oq}\| - d_{ls}$, and a positive number α_{ls}, we consider the following CBF-based constraint,

$$G^{ls}_{ijo}(x, x^*_{ijo}, v) := \dot{h}^{ls}_{ijo}(x, x^*_{ijo}, v) + \alpha_{ls} h^{ls}_{ijo}(x, x^*_{ijo}) \geq 0, \quad \forall j \in \mathcal{N}^{\sigma}_i. \tag{37}$$

We define the velocity of point q as

$$v_q := (1-\lambda^*)v_i + \lambda^* v_j, \tag{38}$$

and its projection onto x_{oq} as

$$\bar{v}_{oq} := \frac{x^T_{oq}}{\|x_{oq}\|} v_q. \tag{39}$$

In the same way as in (23), we consider the following constraint accounting for a safety margin in a scenario where both robots i and j change the velocity of the LOS towards x^*_{ijo} from $\bar{v}_{oq}$ to 0 using the maximum deceleration η.

$$h^{ls}_{ijo}(x, x^*_{ijo}) + \bar{v}_{oq}(t) T_{qo} + \int_0^{T_{qo}} \eta\tau \, \mathrm{d}\tau \geq 0, \tag{40}$$

where $T_{qo} := \frac{0-\bar{v}_{oq}(t)}{\eta}$. From (40), a constraint $\bar{h}^{ls}_{ijo}(x, x^*_{ijo}, v) \geq 0$ is obtained, where

$$\bar{h}^{ls}_{ijo}(x, x^*_{ijo}, v) := \sqrt{2\eta h^{ls}_{ijo}} + \frac{x^T_{oq}}{\|x_{oq}\|} v_q. \tag{41}$$

A derivative condition can be derived for (41) as follows,

$$\left(\nabla_{v_i} \bar{h}^{ls}_{ijo}\right)^T u_i + \left(\nabla_{v_j} \bar{h}^{ls}_{ijo}\right)^T u_j + b^{ls}_{ijo} \geq 0, \tag{42}$$

where $b^{ls}_{ijo} := (\nabla_{x_i} \bar{h}^{ls}_{ijo})^T v_i + (\nabla_{x_j} \bar{h}^{ls}_{ijo})^T v_j + \alpha_{ls} (\bar{h}^{ls}_{ijo})^3$. The constraint in (42) is distributed as follows:

$$\left(\nabla_{v_i} \bar{h}^{ls}_{ijo}\right)^T u_i + \lambda^* b^{ls}_{ijo} \geq 0, \tag{43}$$

$$\left(\nabla_{v_j} \bar{h}^{ls}_{ijo}\right)^T u_j + (1-\lambda^*) b^{ls}_{ijo} \geq 0. \tag{44}$$

Thus, the constraint for robot i to maintain the LOS preservation constraint in (17) is described as

$$A^{ls}_{ijo} u_i + \lambda^* b^{ls}_{ijo} \geq 0, \tag{45}$$

where $A^{ls}_{ijo} := (\nabla_{v_i} \bar{h}^{ls}_{ijo})^T$.

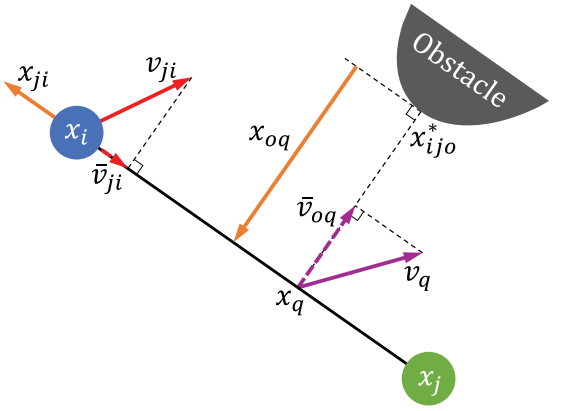

Figure 2. Relative position and velocity between robots i and j, and illustration of point q on LOS.

4. Control algorithm

The outline of the control algorithm for robot i is described as follows.

Step 1: The indicator function $\sigma_{ij}(t)$ for all $j \in \mathcal{N}_i(t)$ is determined to select links in $\mathcal{G}_m$ to be preserved.

Step 2: The desired control input is determined by considering the CBF-based constraints and APFs.

Step 3: If the desired control input violates CBF-based constraints, it is minimally modified to obtain the actual control input u_i.

Each step of the algorithm is described in detail in the following subsections. In Step 1, we use the same algorithm as in [5]. The main difference from [5] is that the CBF-based constraints described in Section 3 are used in Steps 2 and 3 instead of APFs.

4.1. Link management (Step 1)

The purpose of link management is to achieve a slender swarm shape with fewer links between robots when navigating narrow passages and to enable the swarm to navigate through dense obstacle-laden environments. To achieve a slender shape with fewer links between robots, it is necessary to deactivate some of the redundant edges in the formation graph $\mathcal{G}_m$. To avoid leaving some robots behind, this deactivation process needs to be performed while preserving connectivity. In this paper, we adopt simple rules proposed in [4, 5] to deactivate edges in a decentralized manner without data transmission, as described below. See [4] for theoretical analysis on connectivity preservation of $\mathcal{G}_\sigma$ after these decentralized deactivation rules are applied.

The region $\mathcal{D}_{ij}$ is defined for x_i and x_j, as shown in Figure 3, to describe the link deactivation rule,

$$\mathcal{D}_{ij} := \{q \mid (q - x_i)^T x_{ij} > 0, \ (q - x_j)^T x_{ij} < 0\}. \tag{46}$$

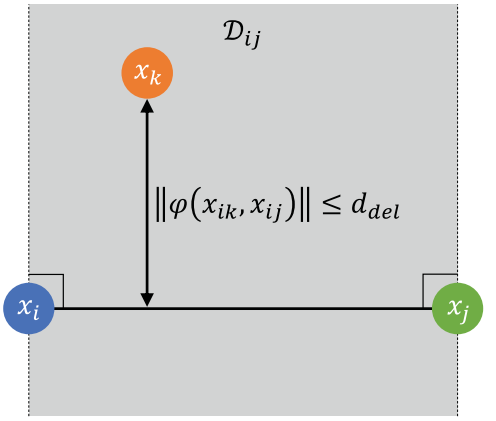

Figure 3. 2D illustration of D_{ij} and $\|\varphi(x_{ik}, x_{ij})\|$.

Furthermore, a projection function $\varphi(p, q)$ is employed to project the vector p onto the plane perpendicular to vector q. The projection function can be defined as follows,

$$\varphi(p, q) := p - \frac{p^T q}{\|q\|^2} q. \tag{47}$$

As shown in Figure 3, the distance between x_k and the line connecting x_i and x_j can be computed as $\|\varphi(x_{ik}, x_{ij})\|$. If robots i, j, and k satisfy the following condition, then the link $(i, j) \in \mathcal{E}_m$ is deactivated:

$$\begin{aligned} &\|\varphi(x_{ik}, x_{ij})\| \leq d_{del},\ x_k \in \mathcal{D}_{ij}, \\ &(i, j) \in \mathcal{E}_m,\ (j, k) \in \mathcal{E}_m,\ (k, i) \in \mathcal{E}_m, \end{aligned} \tag{48}$$

where d_{del} is a positive constant satisfying $d_{del} < d_c \sin \frac{\pi}{3}$.

Another rule is that if only one edge of a connected robot triangle has a length of d_m, then that edge will be deactivated. Specifically, the link (i, j) will be deactivated if robots i, j, and k satisfy the following condition,

$$\begin{aligned} &d_m - \delta_m < \|x_{ij}\| \leq d_m,\ \|x_{kj}\| < d_m - \delta_m, \\ &\|x_{ik}\| < d_m - \delta_m,\ (i, j) \in \mathcal{E}_m,\ (j, k) \in \mathcal{E}_m,\ (k, i) \in \mathcal{E}_m, \end{aligned} \tag{49}$$

where δ_m is a small positive number relative to d_m. In summary, σ_{ij} is defined as follows:

$$\sigma_{ij} = \begin{cases} 0, & \text{if (48) or (49)} \\ 1, & \text{otherwise} \end{cases}. \tag{50}$$

4.2. Desired control input calculation (Step 2)

Since the leader is given a target path, the desired control input for the leader is determined to follow that path. In simulations and experiments, the following simple controller is used for a sequence of waypoints on the target path:

$$a_N = k_p(x_{wp} - x_N), \quad k_p > 0, \tag{51}$$

where x_{wp} is the next waypoint on the target path.

Determining desired inputs for the follower robots is more challenging because they do not have knowledge of the target path. The desired control input for follower robots, a_i, is expressed by the following equation,

$$a_i = a_i^{con} + a_i^{da} + a_i^{ag}, \tag{52}$$

where a_i^{con} has four components as follows,

$$a_i^{con} = a_i^m + a_i^c + a_i^{ob} + a_i^{ls}. \tag{53}$$

The significance of each component in (52) and (53) is elucidated in Table 1. It should be noted that although CBF-based constraints are imposed in Step 3, a_i^{con} in Step 2 is necessary for the following reasons: First, merely satisfying the given constraints in Step 3 is not enough for robots to pass through narrow spaces without getting stuck even if the link deactivation in Step 1 is applied. Second, robots move extremely slowly without a_i^{con}. Since Step 3 makes only the minimum change to a desired input a_i in (52) so as to satisfy the CBF-based constraints, distances between robots tend to become close to the maximum limit so that the leader needs to wait for the followers to satisfy the maximum distance constraint. Thus, the purpose of a_i^{con} is not only to satisfy the constraints but also to generate accelerations to move followers towards the leader without getting stuck in narrow spaces.

In the previous work [5], desired inputs were determined by employing repulsive APFs based on specified constraints and an attractive APF to foster group cohesion. However, the repulsive APFs caused oscillatory behaviors in the desired inputs, as mentioned in Section 3.1. In this paper, we use repulsive functions based on CBFs to compute a_i^{con} instead of APFs.

The acceleration component a_i^m is determined using the following repulsive force to satisfy the constraint in (30)

$$a_i^m = \frac{1}{n_i^\sigma} \sum_{j \in \mathcal{N}_i^\sigma} \omega_m(G_{ij}^m)\, \nabla_{v_i} G_{ij}^m, \tag{54}$$

Table 1. Description of acceleration components.

a_i^m	Acceleration to enforce maximum link length
a_i^c	Acceleration to avoid collision with robots
a_i^{ob}	Acceleration to avoid collision with obstacles
a_i^{ls}	Acceleration to preserve LOS between robots
a_i^{da}	Acceleration to avoid deadlock
a_i^{ag}	Acceleration to make the group more cohesive

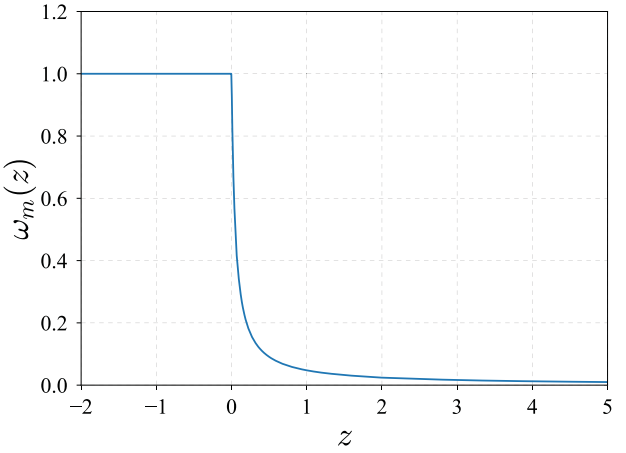

Figure 4. Example of $\omega_m(z)$ for $\mu_m = 1, \beta_m = 0.1$.

where n_i^σ denotes the cardinality of the set $\mathcal{N}_i^\sigma$, and ω_m is defined as

$$\omega_m(z) := \frac{\mu_m \, \beta_m}{|z| + z + \beta_m}, \quad \mu_m > 0, \ \beta_m > 0. \tag{55}$$

Figure 4 illustrates an example of the function $\omega_m(z)$ for $\mu_m = 1, \beta_m = 0.1$. Here, the variable μ_m establishes the maximum value of the function, while β_m determines the rate at which $\omega_m(z)$ decreases for $z \geq 0$. The purpose of a_i^m in (54) is to increase G_{ij}^m when the constraint $G_{ij}^m \geq 0$ in (21) is violated ($G_{ij}^m < 0$) or on the verge of violation ($G_{ij}^m \simeq 0$). From (21), we have

$$\begin{aligned}\dot{G}_{ij}^m(x, v, u_i, u_j) = (\nabla_{x_i} G_{ij}^m)^T v_i + (\nabla_{v_i} G_{ij}^m)^T u_i \\ + (\nabla_{x_j} G_{ij}^m)^T v_j + (\nabla_{v_j} G_{ij}^m)^T u_j.\end{aligned} \tag{56}$$

To increase G_{ij}^m, u_i must make the second term of $\dot{G}_{ij}^m$ a positive large value. If $u_i = a_i^m$ is applied, the second term is positive except in the case of $\nabla_{v_i} G_{ij}^m = 0$. Furthermore, the weighting coefficient $\omega(G_{ij}^m)$ becomes large when $G_{ij}^m < 0$ or $G_{ij}^m \simeq 0$ as shown in Figure 4 so that the second term of $\dot{G}_{ij}^m$ becomes large.

Similarly, a_i^c is determined through the following repulsive force to satisfy (21)

$$a_i^c = \frac{1}{n_i} \sum_{j \in \mathcal{N}_i} \omega_c(G_{ij}^c) \, \nabla_{v_i} G_{ij}^c, \tag{57}$$

using $\omega_c(z)$ defined in the same way as (55) for constants μ_c and β_c. n_i denotes the cardinality of set $\mathcal{N}_i$.

To define a_i^{ob}, we specify the set $\mathcal{O}_i^{ob}$ comprising the nearest obstacle points detected from robot i,

$$\mathcal{O}_i^{ob} = \underset{x_o \in \mathcal{O}_i}{\operatorname{argmin}} \, \|x_i - x_o\|. \tag{58}$$

The accelerations to avoid collision with obstacles is determined by:

$$a_i^{ob} = \frac{1}{n_i^{ob}} \sum_{x_o \in \mathcal{O}_i^{ob}} \omega_{ob}(G_{io}^{ob}) \, \nabla_{v_i} G_{io}^{ob}. \tag{59}$$

where $\omega_{ob}(z)$ is defined in the same way as in (55) for constants μ_o and β_o.

To elaborate on the term a_i^{ls}, we define the set $\mathcal{LS}_i$, which represents the neighboring robot and the corresponding obstacle that are nearest to the boundary of the LOS preservation condition in (17), defined as follows:

$$\mathcal{LS}_i = \underset{j \in \mathcal{N}_i^\sigma, \, x_o \in \mathcal{O}_i \cap \mathcal{D}_{ij}}{\operatorname{argmin}} \|\varphi(x_{io}, x_{ij})\|. \tag{60}$$

The acceleration to preserve LOS is determined by the following equation,

$$a_i^{ls} = \frac{1}{n_i^{ls}} \sum_{(j, x_{ijo}^*) \in \mathcal{LS}_i} \omega_{ls}(G_{ijo}^{ls}) \, \nabla_{v_i} G_{ijo}^{ls}. \tag{61}$$

A deadlock situation may occur when three robots forming an isosceles triangle move into a narrow space, as illustrated in Figure 5. In this scenario, the robots cannot find the correct link to deactivate without losing connectivity; thus the group will get stuck without being able to enter into the narrow space. To prevent this situation, the longest edge of the triangle formed by the three robots is made longer than the other edges. To further address this issue, we introduce an acceleration component, a_i^{da}, that aims to extend the longest edge of a triangle consisting of robot triplets i, j, and k. Specifically, if there is an obstacle between either $(i, j) \in \mathcal{E}_m$ or $(j, k) \in \mathcal{E}_m$, and $(i, k) \in \mathcal{E}_m$, and $\|x_{ji}\| > \|x_{jk}\| > d_c + \delta_c, d_c + \delta_c > \|x_{ik}\| > d_c$, then robot i will generate an acceleration to extend the longest edge (i, j) of the triangle further than the other edges. We denote the set of robots j corresponding to long edges as $\mathcal{DA}_i$. To enforce the deadlock avoidance constraint, the acceleration a_i^{da} is determined as

$$a_i^{da} = \frac{\beta_{da}}{n_i^{da}} \sum_{j \in \mathcal{DA}_i} \frac{x_{ji}}{\|x_{ji}\|}, \quad \beta_{da} > 0, \tag{62}$$

where n_i^{da} denote the cardinality of set $\mathcal{DA}_i$. The acceleration component a_i^{da} will generate an acceleration when the lengths of the two longest links in a robot triplet converge towards equal lengths, thereby averting the isosceles triangle shape.

To maintain cohesion among the robots, we introduce the following attractive APF $\Psi_i^{ag}(x)$, which promotes cohesion among the robots in the absence of obstacles.

$$\Psi_i^{ag}(x) = \frac{1}{n_i^{coh}} \sum_{j \in \mathcal{S}_i \setminus \mathcal{N}_i^\sigma} \phi^{coh}(\|x_{ji}\|)$$

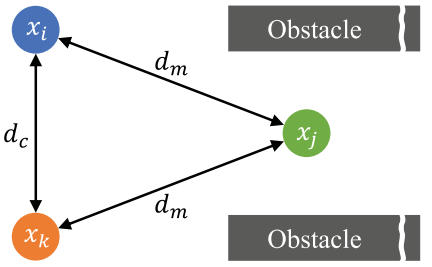

Figure 5. 2D illustration of deadlock isosceles triangle.

$$+ \sum_{(j,k)\in\mathcal{CB}_i} \phi^{cb}(\|\varphi(x_{ij}, x_{kj})\|), \tag{63}$$

$$\phi^{coh}(z) = \begin{cases} \frac{1}{2}(z-d_m)^2, & z > d_m,\ \mathcal{O}_i = \emptyset \\ 0, & \text{otherwise} \end{cases}, \tag{64}$$

$$\mathcal{CB}_i = \{(j,k) \in \mathcal{N}_i^\sigma \mid \|x_{kj}\| > d_m\}, \tag{65}$$

$$\phi^{cb}(z) = \begin{cases} \frac{(d_{del}+\delta_{del}-z)^2}{2}, & z < d_{del}+\delta_{del},\ \mathcal{O}_i = \emptyset \\ 0, & \text{otherwise.} \end{cases} \tag{66}$$

The ϕ^{coh} component of Ψ_i^{ag} motivates robot i to approach the robots within its sensing range, denoted by the set $\mathcal{S}_i$. This cohesive behavior is achieved by minimizing the distance between robot i and the robots in $\mathcal{S}_i \setminus \mathcal{N}_i^\sigma$, which represents the set of robots that have a direct LOS to robot i but are not included in $\mathcal{N}_i^\sigma$. The cardinality of $\mathcal{S}_i \setminus \mathcal{N}_i^\sigma$ denoted as n_i^{coh}. On the other hand, the ϕ^{cb} component of Ψ_i^{ag} disrupts the straight-line formation between robot i and its neighboring robots, facilitating the quick creation of links to them and establishing new LOS for them. The ϕ^{cb} component is defined based on the distance between robot i and the nearest point on the line $\mathcal{L}(x_j, x_k)$, where $d_{del} + \delta_{del}$ represents the intended distance margin for establishing new LOS. The $\mathcal{CB}_i$ set is used to determine the ϕ^{cb} component of the APF, encompassing the pair of robots linked with robot i and separated by a distance greater than the maximum link distance d_m. The acceleration to enforce aggregation is determined as follows, where β_{ag} is a design constant,

$$a_i^{ag} = -\beta_{ag}\nabla_{x_i}\Psi_i^{ag}(x). \tag{67}$$

Overall, the APF $\Psi_i^{ag}(x)$ enables the follower robots to gather around the leader robot, thus preventing the swarm from dividing and allowing for better coordination among all robots.

4.3. Verification of constraint enforcement (Step 3)

For the desired input a_i determined in Step 2, the control input u_i is calculated by solving a numerical optimization problem considering the given CBF constraints. Since multiple constraints are severely imposed in our control problem, the control input satisfying all the constraints does not necessarily exist. In this paper, a classical approach utilizing soft bounds [25] is adopted to ensure the feasibility of the optimization problem.

To describe the optimal control problem in a compact form, we describe the CBF-based constraints in Section 3 as follows:

$$Au_i + b \geq 0, \tag{68}$$

where the inequality of each row is one of the constraints in (29), (31), (34), and (45).

Robot i $(= 1, \ldots, N)$ determines u_i by solving the following optimization problem at each time step,

$$\min_{u_i\in\mathbb{R}^n,\ \epsilon\in\mathbb{R}} \{\|u_i - a_i\|^2 + \rho\epsilon\}$$

subject to

$$\begin{aligned} Au_i + b + \epsilon\mathbf{1} &\geq 0, \\ \|u_i\| &\leq \eta, \\ \epsilon &\geq 0, \end{aligned} \tag{69}$$

where $\mathbf{1}$ is a vector with all entries being one, and ϵ is a slack variable which is non-zero only if the constraint in (68) is violated. Thus, ϵ indicates the amount of constraint violation. By heavily penalizing ϵ in the cost function using a large weight $\rho > 0$, the optimal solution to the problem in (69) tries to minimize constraint violations as much as possible. This optimization is reduced to a semi-definite programming problem and thus can be solved efficiently.

4.4. Recovery control

In Step 3 of the algorithm, we relax the CBF-based constraints to obtain a control input. This relaxation may lead to violations of the original position-based constraints defined in Section 2. To address this issue, a recovery control law is introduced to manage constraint violations and return the robot to a safe zone. The recovery control law is activated when one or more constraints are violated. In such cases, the control input for the robot is determined solely by computing acceleration for the violated constraints, disregarding the other acceleration components. More specifically, the recovery control input is computed as follows,

$$u_i = a_i^r - k_r v_i, \tag{70}$$

$$a_i^r = c_m a_i^{mr} + c_c a_i^{cr} + c_{ob} a_i^{obr} + c_{ls} a_i^{lsr}, \tag{71}$$

$$a_i^{mr} = \sum_{j \in \mathcal{R}_i^m} \frac{\|x_{ij}\| - d_m}{\bar{d}_m - d_m} \frac{x_{ij}}{\|x_{ij}\|}, \tag{72}$$

$$a_i^{cr} = \sum_{j \in \mathcal{R}_i^c} \frac{\|x_{ij}\| - d_c}{d_c - \underline{d}_c} \frac{x_{ij}}{\|x_{ij}\|}, \tag{73}$$

$$a_i^{obr} = \sum_{x_o \in \mathcal{R}_i^{ob}} \frac{\|x_{io}\| - d_o}{d_o - \underline{d}_o} \frac{x_{io}}{\|x_{io}\|}, \tag{74}$$

$$a_i^{lsr} = \sum_{(j, x_{ijo}^*) \in \mathcal{R}_i^{ls}} \frac{\|x_{qo}\| - d_{ls}}{d_{ls} - \underline{d}_{ls}} \frac{x_{qo}}{\|x_{qo}\|}, \tag{75}$$

$$x_{qo} = -\varphi(x_{ijo}^* - x_i, x_{ij}), \tag{76}$$

where $c_m > 0, c_c > 0, c_{ob} > 0, c_{ls} > 0$ are weights assig ned for each recovery acceleration component. Here, $\mathcal{R}_i^m$ denotes the robots that violate condition (14), while $\mathcal{R}_i^c$ refers to robots in violation of condition (15). The set $\mathcal{R}_i^{ob}$ includes obstacle points that breach condition (16), and $\mathcal{R}_i^{ls}$ comprises both robots and the corresponding obstacle points that violate the LOS constraint specified in (17).

5. Non-optimization-based method

Although the method in Section 4 requires to solve a numerical optimization problem at each time step in Step 3, robots may not always possess sufficient computational capacity for this task in some cases. Thus, many control methods for multi-agent systems adopt simple control algorithms without solving numerical optimization [4, 5, 26–30]. These methods determine control input by employing a combination of repulsive and attractive acceleration components based on APFs. In other words, high priority is given to constraints with large repulsive acceleration, while constraints with small repulsive acceleration are ignored.

In this section, we presents a new non-optimization-based method based on CBF constraints. The proposed method is based on a combination of repulsive and attractive acceleration components in the same way as in the existing methods [4, 5, 26–30]. A significant difference from those existing methods is that repulsive acceleration components are constructed using CBF-based constraints rather than APFs. More specifically, the proposed non-optimization-based method determines the direction of the control input using a_i in (52), which is the sum of repulsive and attractive acceleration components. Then, the magnitude of the control input is adjusted, as follows:

$$u_i = \lambda \frac{a_i}{\|a_i\|}, \quad \lambda \in [0, \eta]. \tag{77}$$

For the rth row of the constraint in (68), $A_r u_i + b_r \geq 0$, we determine an allowable region $[\lambda_r^{\min}, \lambda_r^{\max}]$ depending on situations as follows:

Case 1 : $A_r \frac{a_i}{\|a_i\|} \geq 0$ and $b_r \geq 0$.
In this case, λ can take any positive value as the condition will always be satisfied. Thus, the limits are set as $\lambda_r^{\min} = 0, \lambda_r^{\max} = \eta$.

Case 2 : $A_r \frac{a_i}{\|a_i\|} < 0$ and $b_r \geq 0$.
In this case, the constraint will be satisfied for $\lambda \leq -\frac{b_r \|a_i\|}{A_r a_i}$. Thus, the limits are set as $\lambda_r^{\min} = 0, \lambda_r^{\max} = -\frac{b_r \|a_i\|}{A_r a_i}$.

Case 3 : $A_r \frac{a_i}{\|a_i\|} \leq 0$ and $b_r < 0$.
In this scenario, the inequality cannot be satisfied for a non-negative λ. Since there is no choice other than to ignore this constraint, the limits are set as $\lambda_r^{\min} = 0, \lambda_r^{\max} = \eta$.

Case 4 : $A_r \frac{a_i}{\|a_i\|} > 0$ and $b_r < 0$.
In this case, the constraint will be satisfied for $\lambda \geq -\frac{b_r \|a_i\|}{A_r a_i}$. Thus, if $\eta < -\frac{b_r \|a_i\|}{A_r a_i}$, the limits are set as $\lambda_r^{\min} = 0, \lambda_r^{\max} = \eta$, since this constraint cannot be satisfied for $\lambda \in [0, \eta]$. Otherwise, the limits are set as $\lambda_r^{\min} = -\frac{b_r \|a_i\|}{A_r a_i}, \lambda_r^{\max} = \eta$.

To determine λ in (77), we use the following $\lambda^{\min}$ and $\lambda^{\max}$

$$\lambda^{\min} := \max_r \lambda_r^{\min}, \tag{78}$$

$$\lambda^{\max} := \min_r \lambda_r^{\max}. \tag{79}$$

If $\lambda^{\min} \leq \lambda^{\max}$, it is easy to determine λ depending on the $\|a_i\|$. On the other hand, $\lambda^{\max} < \lambda^{\min}$ implies that constraints in Cases 2 and 4 conflict with each other. In such cases, we ignore $\lambda^{\min}$ for the following reason: The value of $\lambda^{\min}$ is typically large when the difference between the direction of a_i and a repulsive acceleration of constraints in Case 4 is large. Since we prioritize constraints with repulsive forces in directions similar to a_i, the priority of constraints in Case 4 with a large $\lambda^{\min}$ is low. Therefore, we ignore $\lambda^{\min}$ if $\lambda^{\max} < \lambda^{\min}$. In summary, we determine λ as follows:

$$\lambda = \begin{cases} \lambda^{\max}, & \|a_i\| \geq \lambda^{\max} \\ \lambda^{\min}, & \|a_i\| < \lambda^{\min} \leq \lambda^{\max} \\ \|a_i\|, & \text{otherwise.} \end{cases} \tag{80}$$

Note that this method ignores constraints which are not satisfied by a positive multiple of a_i as in (77). Then, λ in (80) is selected so that $\|u_i - a_i\|$ is minimized subject to the allowable region $\lambda_{\min} \leq \lambda \leq \lambda_{\max}$ for the constraints that are not ignored.

Table 2. Parameters used in the simulation.

d_s	$\bar{d}_m$	d_m	$\underline{d}_c$	d_c	$\underline{d}_o$	d_o
2	1.9	1	0	0.1	0	0.1
$\underline{d}_{ls}$	d_{ls}	d_{del}	δ_m	δ_{del}	α_m	α_c
0	0.05	0.05	0.05	0.05	0.1	0.1
α_{ob}	α_{ls}	β_m	β_c	β_{ob}	β_{ls}	β_{da}
0.4	0.2	0.01	0.01	0.001	0.001	0.01
β_{ag}	ρ	c_m	c_c	c_{ob}	c_{ls}	η
0.5	10^6	1	1	0.4	0.5	1

6. Simulations

We applied the proposed method to a quadrotor model given in [34] with inner-loop PD controllers for attitude and velocity. The simulations were run on a computer with an Intel Core i9 12900k processor, 32 GB of RAM, and an NVIDIA GeForce RTX 3090 graphics card. The control input was computed at intervals of $\Delta t = 0.1$ s, and the system parameters were set as listed in Table 2. The control input u_i determined by the proposed method was passed as the target velocity $v_i(t + \Delta t) = v_i(t) + \Delta t u_i(t)$ to the quadrotor model. Furthermore, considering the maximum acceleration limit, we set $\mu_m = \mu_c = \mu_{ob} = \mu_{ls} = \eta$. Additionally, we limited the maximum speed of the leader to 0.1 m/s, enabling the follower robots to aggregate around the leader in obstacle-free environments.

The parameters of the controllers were determined taking into account the following points. As shown in [10], for smaller values of α_{ob}, robots tend to take larger margins from the limit of the obstacle avoidance constraint in (16). It is expected that α_m, α_c, and α_{ls} exhibit the same behavior for the constraints in (14), (15), and (17), respectively. Thus, if these parameters are too small, too large margins may negatively impact the satisfaction of other constraints. Particularly, too small values of α_{ob} and α_{ls} make it difficult for robots to go into narrow tunnels due to too large margins of distance from walls. Similarly, if large values of β_m, β_c, β_{ob}, and β_{ls} are selected, large repulsive accelerations for CBF-based constraints possibly cause a negative effect to satisfy other constraints. Large values of β_{ob} and β_{ls} make it difficult for robots to go into narrow tunnels due to strong repulsion from the walls. For large values of c_m, c_c, c_{ob} and c_{ls}, accelerations to recover from constraints could generate a speed large enough to violate other constraints right after recovery, particularly in narrow tunnels. A small value should be chosen for β_{da}, since otherwise a large velocity moving a robot in the opposite direction to the leader is generated so that the entire group of robots slow down. For the weighting coefficient of the attractive APF, β_{ag}, a too large value could cause oscillatory behaviors of robots particularly when part of robots are still inside the tunnel. The values of d_{del}, δ_m, and δ_{del} were chosen smaller than d_c without parameter tuning. For ρ, a large value was chosen without tuning.

We used the obstacle-laden environment configuration depicted in Figure 6. The leader is given a target path that passes through the center of the tunnel, starting from the origin, as illustrated by the thick solid line. The obstacle-laden environment is designed as a polyhedron tunnel, with plane obstacles positioned on all four sides of the path (left, right, top, bottom). To define the 3D relative angle between the $(n+1)$th line segment and the nth line segment, we use a pair of angles, denoted as $(\theta_n \neq 0, \gamma_n \neq 0)$ for $n = 1, \ldots, M+1$. Here, θ_n represents the azimuth angle of the $(n+1)$th line segment with respect to the nth line segment, and γ_n represents the elevation angle of the $(n+1)$th line segment with respect to the plane perpendicular to the nth line segment. Thus, M represents the number of corners in the target path. The first and last line segments were divided into parts inside and outside the obstacle area, with l_s and l_e representing the lengths of the parts from the origin to the center of the first obstacle segment and from the center of the last segment to the end of the target path, respectively. Additionally, the length of the nth line segment inside the obstacle area is defined as l_n. The minimum distance between an obstacle and the leader's target path is denoted as ζ.

For the tunnel obstacle in Figure 6, $\mathcal{O}_i$ is defined as the union of polygons forming the tunnel. The nearest point on each polygon from robot i is calculated. Then, among those points, the nearest point within the sensing range of robot i are selected as the elements of $\mathcal{O}_i^{ob}$ in (58). Similarly, for each polygon, the nearest obstacle point within the region $\mathcal{D}_{ij}$ defined in (46) and closest to

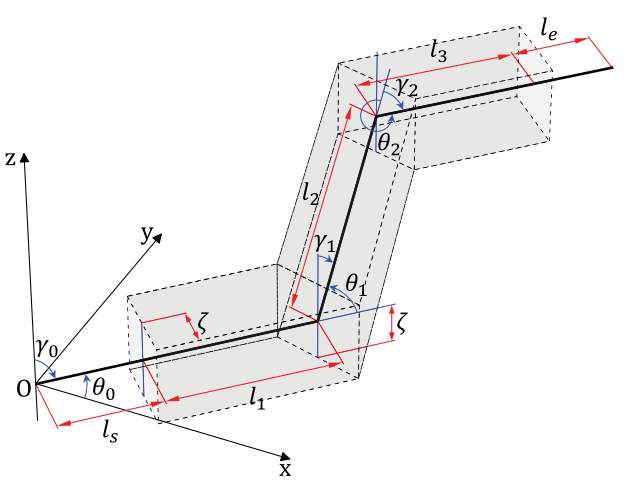

Figure 6. Square tunnel obstacle configuration.

the LOS between robot pairs is determined. The corresponding robot and obstacle point pair are then included as elements of $\mathcal{LS}_i$ in (60).

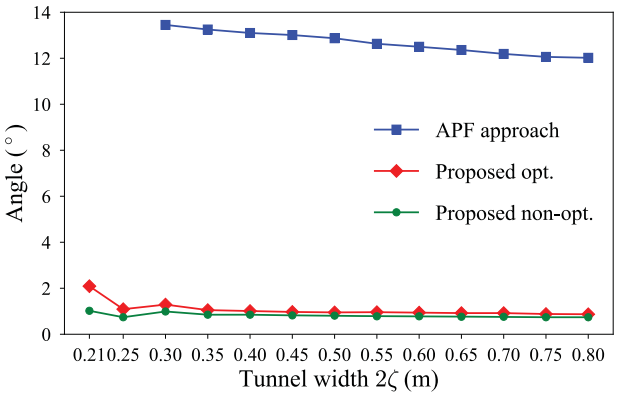

Figure 7. Mean angle differences between consecutive target velocities for different path widths.

First, simulations were conducted for a single-segment tunnel obstacle ($M = 0$) with various path widths (2ζ). Other parameters were set as $\theta_0 = 0, \gamma_0 = \pi/2, l_s = 0.5, l_1 = (N-1)d_m/2$, and $l_e = 20$. For each path-width, a total of 25 sets of simulation trials were conducted by placing $N = 10$ robots in different initial positions. Since the allowable minimum distance between a robot and an obstacle is $d_o = 0.1$ m, a ζ value below 0.1 m was not feasible. Therefore, we performed simulations for cases where ζ ranged from 0.105 m to 0.4 m.

To analyze the oscillatory movement, the angle difference between control inputs at consecutive time steps of the follower robots was calculated. Figure 7 represent the mean angle difference for each path width of the tunnel, with data collected from simulation trials. The analysis was conducted from the start of the simulation until the last robot passed through the tunnel. The results of the APF method for $2\zeta = 0.21$ and 0.25 are not shown,

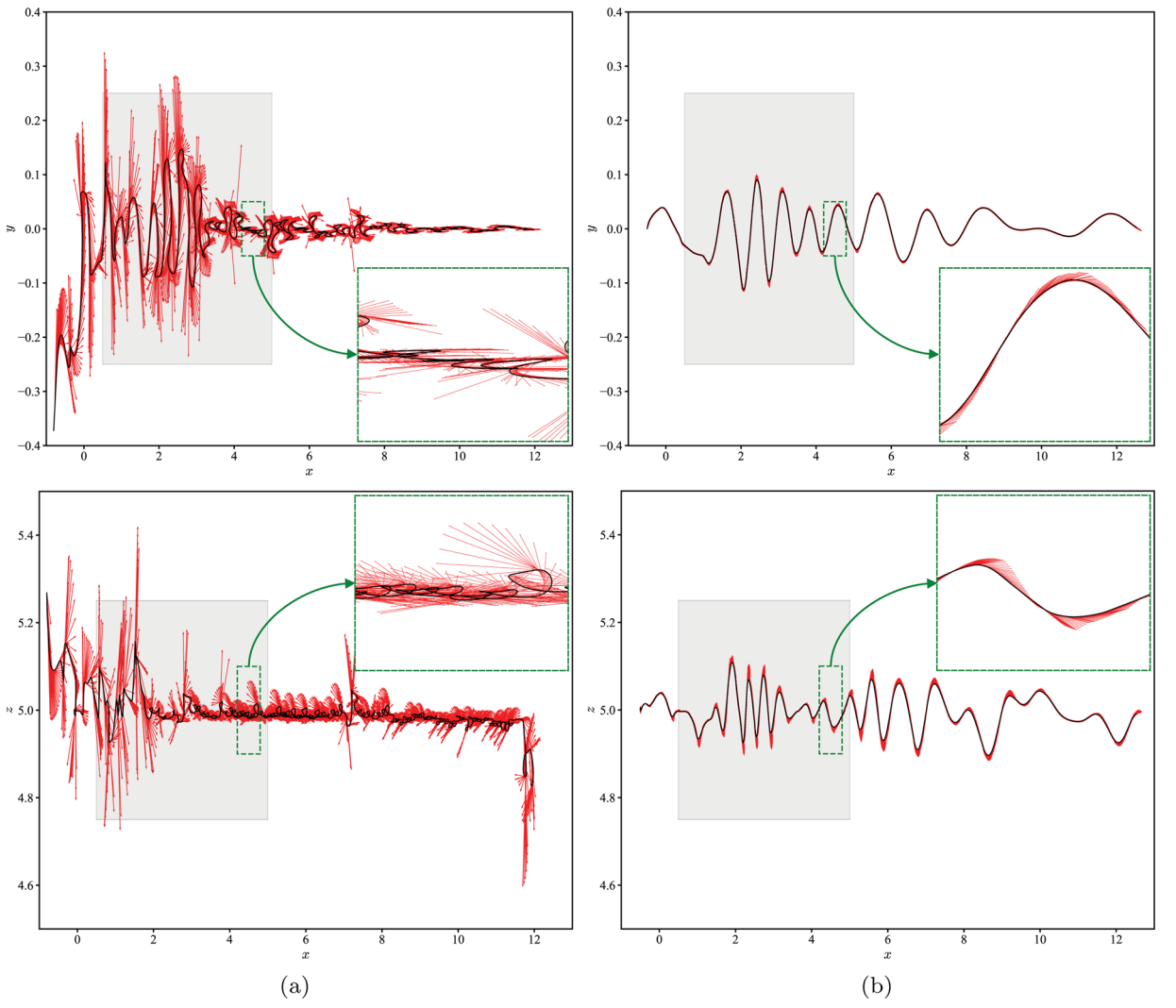

Figure 8. Executed trajectory of the first follower robot: (a) APF approach, and (b) Proposed non-optimization-based method. The red arrows indicate the direction of target velocity commands sent to the quadrotor model, while the black line represents the executed trajectory. The top row shows the trajectories in the *XY*-plane, and the bottom row shows the trajectories in the *XZ*-plane. The obstacle is represented by the gray-shaded area.

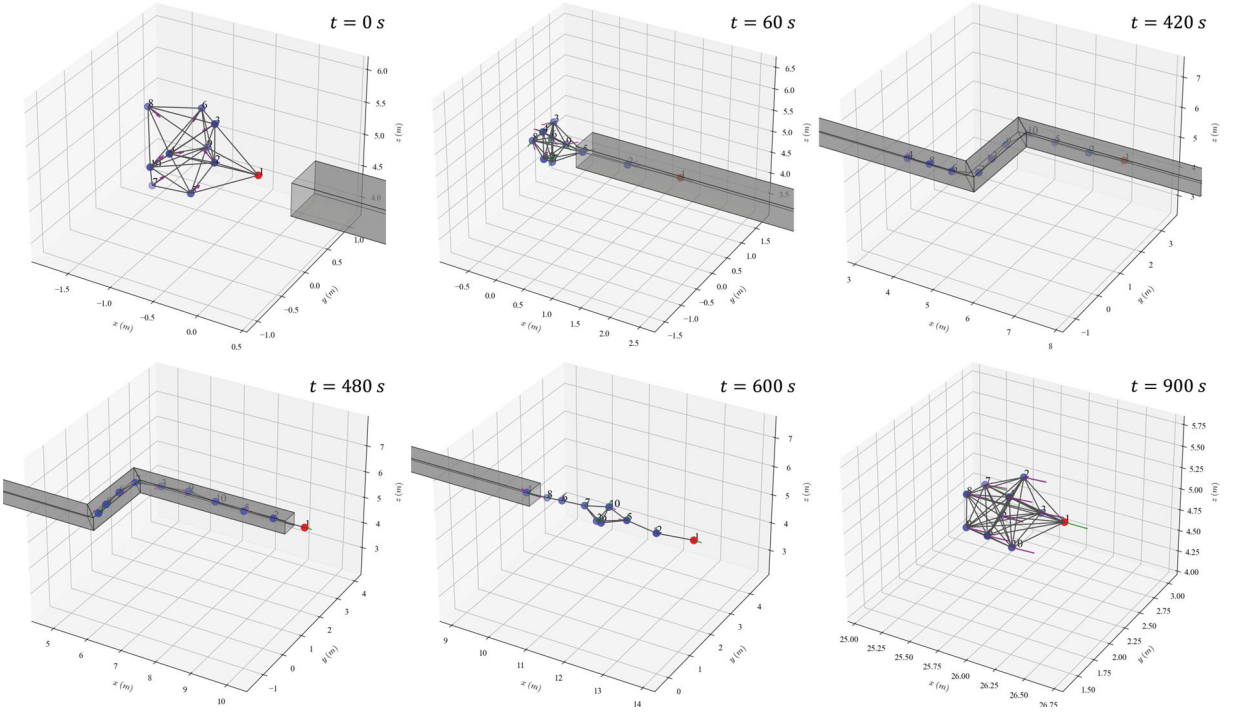

Figure 9. Simulation snapshots of a random trial for the proposed non-optimization-based method in obstacle configuration 3 with $2\zeta = 0.5$ m. The leader and follower robots are represented by red and blue circles, respectively, while the black line segment represents the LOS connectivity between the robots in $\mathcal{G}_\sigma$.

since the robots were unable to pass through the tunnel successfully. This failure occurred for either of two reasons: the robots got stuck in the tunnel or some of them were left behind since the connectivity of the sensing network $\mathcal{G}_s$ was lost. The proposed approaches exhibit a low mean, indicating consistent direction changes in consecutive control inputs and thus minimal vibratory behavior in the robots. In contrast, the APF-based approach shows a high mean, indicating inconsistent direction changes and significant oscillatory movement of the robots.

A typical example of responses that indicate differences in oscillation is illustrated in Figure 8 for the first follower robot. The oscillatory behavior of the robots in the APF approach is clear, as the target velocity commands are arbitrary and inconsistent. In contrast, the proposed approach demonstrates a steady and minimal change in the target velocity command.

Table 3 compares constraint violations, while Table 4 shows the mean computation times for Step 3 between the optimization-based and non-optimization-based methods. To assess the constraint violations, we tallied for each robot the time steps in which any of the constraints in (14)–(17) were violated, across all 25 simulation trials. We then calculated the percentage of these violated time steps relative to the total number of time steps for each tunnel widths. As expected, the non-optimization-based method shows a slightly higher rate of constraint violations compared to the optimization-based method, albeit with much shorter computation times, as illustrated in Table 4.

Table 3. Rate of constraint violation in percentage (%) for single segment tunnel obstacle.

Tunnel width (2ζ)	APF approach	Proposed opt.	Proposed non-opt.
0.21	–	2.2×10^{-3}	2.7×10^{-2}
0.25	–	5.9×10^{-4}	1.3×10^{-2}
0.30	4.2	0	7.4×10^{-4}
0.35	2.9	0	4.3×10^{-4}
0.40	2.0	0	6.0×10^{-4}
0.45	1.8	0	1.3×10^{-4}
0.50	1.7	0	9.4×10^{-4}
0.55	1.6	0	4.3×10^{-4}
0.60	1.6	0	0
0.65	1.6	0	0
0.70	1.6	0	6.0×10^{-4}
0.75	1.6	0	0
0.80	1.6	0	0

It should be noted that as mentioned in Section 4.2, a_i^{con} in (53) plays a crucial role in the proposed methods. Table 5 illustrates a comparison of moving speeds of robots with and without a_i^{con} for the single-segment tunnel, using the optimization-based controller across 25 simulation trials. Without a_i^{con}, the robots failed to pass through the tunnels in all the trials for $2\zeta \leq 0.55$ within the maximum simulation time limited to 36,000 time

Table 4. Mean CPU time in milliseconds for Step 3 in single segment tunnel obstacle for the proposed methods.

Tunnel width (2ζ)	Opt.	Non-opt.
0.21	23.75	0.07
0.25	29.78	0.06
0.30	25.42	0.07
0.35	31.75	0.06
0.40	27.51	0.07
0.45	33.50	0.06
0.50	29.05	0.08
0.55	32.85	0.06
0.60	29.51	0.08
0.65	32.41	0.06
0.70	29.68	0.08
0.75	32.30	0.06
0.80	29.54	0.08

Table 5. Comparison with and without a_i^{con} in (53) for the single-segment tunnel, using the optimization-based controller across 25 simulation trials.

	Number of completed trials		Average time steps taken	
Tunnel width (2ζ)	With a_i^{con}	Without a_i^{con}	With a_i^{con}	Without a_i^{con}
0.21	25	0	8609	–
0.25	25	0	5455	–
0.30	25	0	4804	–
0.35	25	0	4147	–
0.40	25	0	3539	–
0.45	25	0	3053	–
0.50	25	0	2701	–
0.55	25	0	2464	–
0.60	25	1	2305	35511
0.65	25	1	2209	33482
0.70	25	2	2135	34586
0.75	25	12	2035	33841
0.80	25	22	1992	32093

Table 6. Complex obstacle configurations.

Config.	M	$l_1, \ldots$	$\theta_1, \ldots$	$\gamma_1, \ldots$
(1)	1	$\frac{(N-1)d_m}{2}, \frac{(N-1)d_m}{4}$	$\frac{\pi}{3}$	$\frac{\pi}{2}$
(2)	1	$\frac{(N-1)d_m}{2}, \frac{(N-1)d_m}{4}$	$\frac{\pi}{2}$	$\frac{\pi}{2}$
(3)	2	$\frac{(N-1)d_m}{2}, \frac{(N-1)d_m}{4}, \frac{(N-1)d_m}{2}$	$\frac{\pi}{2}, -\frac{\pi}{2}$	$\frac{\pi}{2}, \frac{\pi}{2}$

steps, while they passed in all the trials with a_i^{con}. Furthermore, although the robots passed through in some trials without a_i^{con} for $2\zeta \geq 0.60$, the required time steps in completed trials were roughly over 15 times more than the trials with a_i^{con}.

Next, we conducted simulations for more complex configurations as listed in Table 6 with $N = 10$ robots. Table 7 presents the constraint violation rates across 25 simulation trials for obstacle configurations 1–3, while Table 8 shows the CPU time taken in Step 3. The results for $2\zeta \geq 0.4$ are shown, since the robots fail to navigate through the tunnel successfully in some

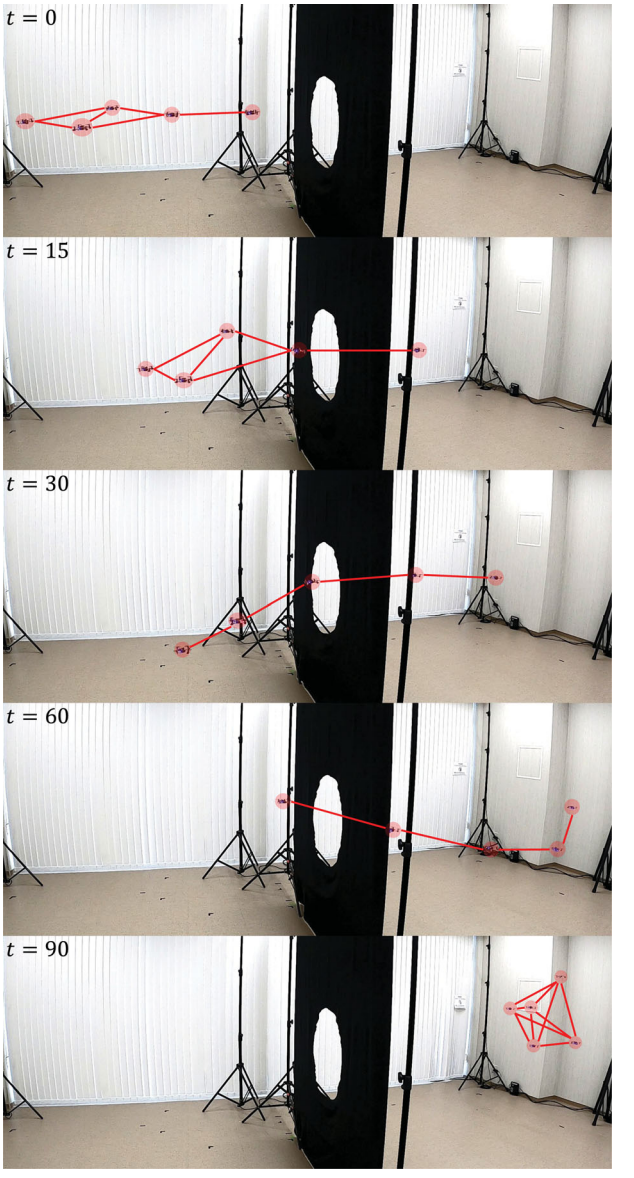

Figure 10. Experiment snapshots of the proposed non-optimization-based method. The red lines denote the edges of the graph $\mathcal{G}_\sigma$.

Table 7. Rate of constraint violation in percentage (%) for obstacle configuration 1–3.

	Config. 1		Config. 2		Config. 3	
Tunnel width (2ζ)	Opt.	Non-opt.	Opt.	Non-opt.	Opt.	Non-opt.
0.4	0	1.0×10^{-2}	0	1.7×10^{-2}	1.3×10^{-4}	9.6×10^{-3}
0.5	0	9.9×10^{-4}	0	2.6×10^{-3}	0	2.5×10^{-3}
0.6	1.3×10^{-3}	0	5.0×10^{-3}	0	3.6×10^{-3}	0
0.7	5.3×10^{-4}	0	4.2×10^{-3}	1.1×10^{-4}	3.5×10^{-3}	1.6×10^{-4}
0.8	0	0	3.7×10^{-3}	1.1×10^{-4}	3.2×10^{-3}	0

Table 8. Mean CPU time in milliseconds for obstacle configuration 1–3.

	Config. 1		Config. 2		Config. 3	
Tunnel width (2ζ)	Opt.	Non-opt.	Opt.	Non-opt.	Opt.	Non-opt.
0.4	27.69	0.07	31.42	0.07	32.09	0.07
0.5	28.84	0.07	31.05	0.08	32.61	0.08
0.6	29.87	0.08	31.53	0.08	31.67	0.08
0.7	30.09	0.08	31.49	0.09	31.67	0.08
0.8	30.74	0.08	31.62	0.09	31.73	0.08

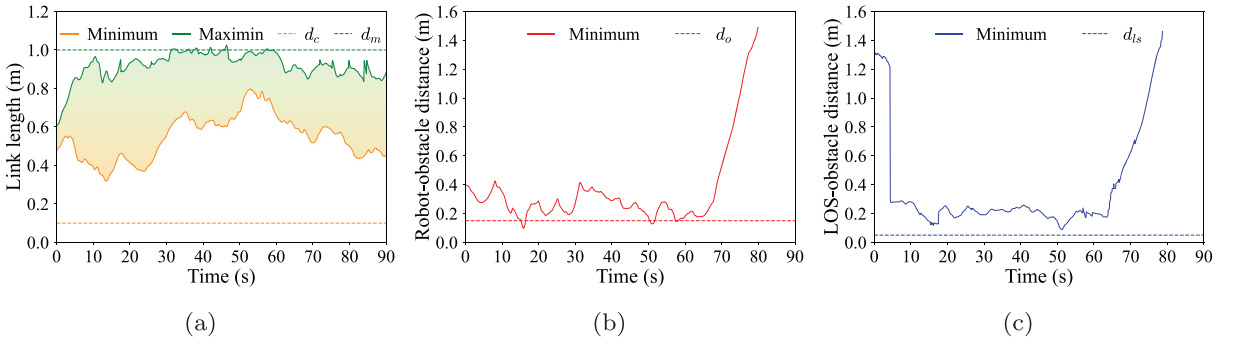

Figure 11. Experiment results: (a) Minimum and maximum distance between the robots in link, (b) Minimum distance between the robot and obstacle, and (c) Minimum distance between the LOS between robots and obstacle at each time instances. The dashed lines in the figures represent the allowable minimum or maximum distances defined in (14)-(17).

cases for $2\zeta \leq 0.3$. These results show that both proposed methods exhibit few constraint violations. However, the computation time for Step 3 is very short for the non-optimization-based method, around 0.08 milliseconds, while the optimization-based method averages around 30.94 milliseconds. Therefore, the proposed non-optimization-based method is expected to be effective in situations where the computer is not fast enough for the optimization-based method to be applied.

Figure 9 displays snapshots of the proposed non-optimization-based method simulation representing a random trial with $2\zeta = 0.5$ m. The snapshots show the successful disconnection of redundant links by the robots upon entering the tunnel, resulting in a chain formation. They also demonstrate the proposed algorithm's ability to allow the robots to navigate sharp turns without losing LOS connectivity while traversing critical corners. Once the robots have passed through the obstacle environment, they effectively aggregate around the leader robot.

7. Experiment

The experiments were conducted in an environment where an obstacle wall was situated in the middle of the leader's target path, featuring a 0.6 m hole in the middle. The positions of the quadrotors were tracked using the OptiTrack Prime17W motion capture system. A formation of five Crazyflie 2.1 quadrotors was employed for experimental validation. Control inputs were computed on a single computer using the proposed method and transmitted to the quadrotors as target velocities with a constant yaw angle. Note that control inputs were calculated in a decentralized manner for each quadrotor based on local information within its sensing range. The wall obstacle is approximated using multiple spheres in the same way as in [5]. The obstacle position data were provided to the robots only when they were within sensing range, as obstacle recognition was not the focus of this experiment. Velocity commands were sent to the robots at every $\Delta t = 0.1$ s, with the same parameters detailed in Table 2 except for $d_s = 1.5, \bar{d}_m = 1.45, d_c = 0.2$, and $d_o = 0.15$.

To deal with the downwash created by the quadrotors, the following constraint is introduced to prevent robots from aligning vertically.

$$\|\bar{x}_i(t) - \bar{x}_j(t)\| \geq d_{dw}, \quad \forall j \in \mathcal{N}_i(t), \tag{81}$$

where $\bar{x}_i$ is defined by making the third element of x_i equal to zero to compute the horizontal distance between robots, and d_{dw} is the minimum allowable distance between robots to prevent the downwash effect. Similar constraints to (28) were derived using a derivative condition as in (21). The parameter values were set as $d_{dw} = 0.2, \alpha_{dw} = 0.5$, and $\beta_{dw} = 10^{-4}$.

Figure 10 presents snapshots of the experiment using the non-optimization-based method in Section 5. Figure 11 illustrate the constraint enforcement status in the experiment. These results show that although the constraints given in (14)–(17) were violated a few times, the robot passed through the obstacle without

collisions by combination with the recovery control in Section 4.4.

8. Conclusion

In this study, we introduced a novel CBF-based control method for guiding a swarm of UAVs through obstacle-rich environments, while maintaining network connectivity without explicit communication between UAVs. The proposed approach addresses the limitations of existing APF-based methods, such as oscillatory behaviors and frequent constraint violations, by utilizing CBF-based constraints instead of repulsive APFs and optimizing control inputs through a numerical optimization problem with soft constraints. Additionally, a non-optimization-based method without numerical optimization was presented. The effectiveness of these methods was validated through extensive simulations and real-world experiments using quadrotors, demonstrating improved performance in reducing vibratory movements compared to APF-based methods. Both proposed approaches showed comparable performance; however, the non-optimization-based method was preferable to the optimization-based method since it required less computation time, making it advantageous for quadrotors with limited computational power. Future work will explore the application of the proposed method in more complex environments involving larger swarms of UAVs equipped with onboard sensing and computation capabilities.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work is supported in part by Japan Society for the Promotion of Science (JSPS) KAKENHI [grant number 22K04171].

Notes on contributors

Thiviyathinesvaran Palani received the B.Sc. Eng. (Hons) in Electrical and Electronics Engineering from South Eastern University of Sri Lanka in 2017 and M.Eng. from Kyoto University of Advanced Science in 2022. He worked as a Lecturer at the Faculty of Engineering, University of Sri Jayewardenepura, Sri Lanka, from 2017 to 2021. He is currently working toward the PhD degree at Kyoto University of Advanced Science, Japan. His research interests include robotics and decentralized control of multi-robot systems.

Hiroaki Fukushima received the B.S. and M.S. degrees in engineering and Ph.D. degree in informatics from Kyoto University, Japan, in 1995, 1998 and 2001, respectively. From 1999 to 2004 he was a Research Fellow of Japan Society for the Promotion of Science. From 2004 to 2009 he worked as a Research Associate and Assistant Professor at the University of Electro-Communications, Japan. From 2009 to 2019 he was a Junior Associate Professor of Kyoto University, Japan. Currently he is a Professor of Kyoto University of Advanced Science, Japan. His research interests include control design of mobile robots, predictive control, and system identification.

Shunsuke Izuhara received the Ph.D. degree in mechanical engineering from Toyohashi University of Technology, Japan, in 2022. He is currently an Assistant Professor with the Faculty of Environmental, Life, Natural Science and Technology, Okayama University, Okayama, Japan. His research interest includes micro piezoelectric actuators.

ORCID

Thiviyathinesvaran Palani http://orcid.org/0000-0002-4863-047X
Hiroaki Fukushima http://orcid.org/0000-0002-0646-6974
Shunsuke Izuhara http://orcid.org/0000-0003-0492-4991

References

[1] Parker LE, Rus D, Sukhatme GS. Multiple mobile robot systems. In: Siciliano B, Khatib O, editors. Springer handbook of robotics. Cham: Springer International Publishing; 2016. Springer Handbooks; p. 1335–1384.
[2] Mesbahi M, Egerstedt M. Graph theoretic methods in multiagent networks. Princeton (NJ): Princeton University Press; 2010.
[3] Dorigo M, Theraulaz G, Trianni V. Swarm robotics: past, present, and future [point of view]. Proc IEEE. 2021;109(7):1152–1165. doi: 10.1109/JPROC.2021.3072740
[4] Sakai D, Fukushima H, Matsuno F. Leader–follower navigation in obstacle environments while preserving connectivity without data transmission. IEEE Trans Control Syst Technol. 2018;26(4):1233–1248. doi: 10.1109/TCST.2017.2705121
[5] Nomura Y, Fukushima H, Matsuno F. Navigation of multiple UAVs in 3d obstacle environments while preserving connectivity without data transmission. In: 2021 IEEE Conference on Control Technology and Applications (CCTA). IEEE; 2021. p. 82–89.
[6] Ames AD, Xu X, Grizzle JW, et al. Control barrier function based quadratic programs for safety critical systems. IEEE Trans Automat Contr. 2017;62(8):3861–3876. doi: 10.1109/TAC.2016.2638961
[7] Wang L, Ames AD, Egerstedt M. Safety barrier certificates for collisions-free multirobot systems. IEEE Trans Robot. 2017;33(3):661–674. doi: 10.1109/TRO.8860
[8] Chen Y, Singletary A, Ames AD. Guaranteed obstacle avoidance for multi-robot operations with limited actuation: a control barrier function approach. IEEE Control Syst Lett. 2021;5(1):127–132. doi: 10.1109/LCSYS.7782633
[9] Ames AD, Coogan S, Egerstedt M, et al. Control barrier functions: theory and applications. In: 2019 18th European Control Conference (ECC). IEEE; 2019. p. 3420–3431.

[10] Singletary A, Klingebiel K, Bourne J, et al. Comparative analysis of control barrier functions and artificial potential fields for obstacle avoidance. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE; 2021. p. 8129–8136.
[11] Origane Y, Hattori Y, Kurabayashi D. Control input design for a robot swarm maintaining safety distances in crowded environment. Symmetry. 2021;13(3):478. doi: 10.3390/sym13030478
[12] Ibuki T, Hirano T, Funada R, et al. Optimization-based distributed safety control with applications to collision avoidance for mobile robotic networks. Adv Robot. 2023;37(1-2):87–98. doi: 10.1080/01691864.2022.2119886
[13] Capelli B, Fouad H, Beltrame G, et al. Decentralized connectivity maintenance with time delays using control barrier functions. In: 2021 IEEE International Conference on Robotics and Automation (ICRA). IEEE; 2021. p. 1586–1592.
[14] Fu J, Wen G, Yu X, et al. Distributed formation navigation of constrained second-order multiagent systems with collision avoidance and connectivity maintenance. IEEE Trans Cybern. 2022;52(4):2149–2162. doi: 10.1109/TCYB.2020.3000264
[15] Luo W, Yi S, Sycara K. Behavior mixing with minimum global and subgroup connectivity maintenance for large-scale multi-robot systems. In: 2020 IEEE International Conference on Robotics and Automation (ICRA). IEEE; 2020. p. 9845–9851.
[16] Özkahraman O, Ögren P. Combining control barrier functions and behavior trees for multi-agent underwater coverage missions. In: 2020 59th IEEE Conference on Decision and Control (CDC). IEEE; 2020. p. 5275–5282.
[17] Notomista G, Ruf SF, Egerstedt M. Persistification of robotic tasks using control barrier functions. IEEE Robot Autom Lett. 2018;3(2):758–763. doi: 10.1109/LSP.2016.
[18] Fouad H, Beltrame G. Energy autonomy for robot systems with constrained resources. IEEE Trans Robot. 2022;38(6):3675–3693. doi: 10.1109/TRO.2022.3175438
[19] Hegde A, Ghose D. Multi-uav collaborative transportation of payloads with obstacle avoidance. IEEE Control Syst Lett. 2021;6:926–931. doi: 10.1109/LCSYS.2021.3087339
[20] Herguedas R, Aranda M, López-Nicolás G, et al. Multi-robot control with double-integrator dynamics and control barrier functions for deformable object transport. In: 2022 International Conference on Robotics and Automation (ICRA). IEEE; 2022. p. 1485–1491.
[21] Sebastián E, Montijano E, Sagüés C. Adaptive multi-robot implicit control of heterogeneous herds. IEEE Trans Robot. 2022;38(6):3622–3635. doi: 10.1109/TRO.2022.3183537
[22] Breeden J, Panagou D. Compositions of multiple control barrier functions under input constraints. In: 2023 American Control Conference (ACC). IEEE; 2023. p. 3688–3695.
[23] Molnar TG, Ames AD. Composing control barrier functions for complex safety specifications. IEEE Control Syst Lett. 2023;7:3615–3620. doi: 10.1109/LCSYS.2023.3339719
[24] Rauscher M, Kimmel M, Hirche S. Constrained robot control using control barrier functions. In: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE; 2016. p. 279–285.
[25] Maciejowski JM. Predictive control with constraints. Harlow: Prentice Hall, Pearson Education Limited; 2002.
[26] Mondal A, Bhowmick C, Behera L, et al. Trajectory tracking by multiple agents in formation with collision avoidance and connectivity assurance. IEEE Syst J. 2018;12(3):2449–2460. doi: 10.1109/JSYST.4267003
[27] Qiao Y, Huang X, Yang B, et al. Formation tracking control for multi-agent systems with collision avoidance and connectivity maintenance. Drones. 2022;6(12):419. doi: 10.3390/drones6120419
[28] Olfati-Saber R. Flocking for multi-agent dynamic systems: algorithms and theory. IEEE Trans Automat Contr. 2006;51(3):401–420. doi: 10.1109/TAC.2005.864190
[29] Su H, Wang X, Lin Z. Flocking of multi-agents with a virtual leader. IEEE Trans Automat Contr. 2009;54(2):293–307. doi: 10.1109/TAC.2008.2010897
[30] Sakai D, Fukushima H, Matsuno F. Flocking for multi-robots without distinguishing robots and obstacles. IEEE Trans Control Syst Technol. 2017;25(3):1019–1027. doi: 10.1109/TCST.2016.2581148
[31] Palani T, Fukushima H, Izuhara S. Control barrier function based decentralized UAV swarm navigation while preserving connectivity without explicit communication. In: 2023 American Control Conference (ACC). IEEE; 2023. p. 1767–1774.
[32] Nguyen Q, Sreenath K. Exponential control barrier functions for enforcing high relative-degree safety-critical constraints. In: 2016 American Control Conference (ACC). IEEE; 2016. p. 322–328.
[33] Xiao W, Belta C. Control barrier functions for systems with high relative degree. In: 2019 IEEE 58th Conference on Decision and Control (CDC). IEEE; 2019. p. 474–479.
[34] Corke P. Robotics, vision and control: fundamental algorithms in matlab. 2nd ed. Cham: Springer International Publishing; 2017.